

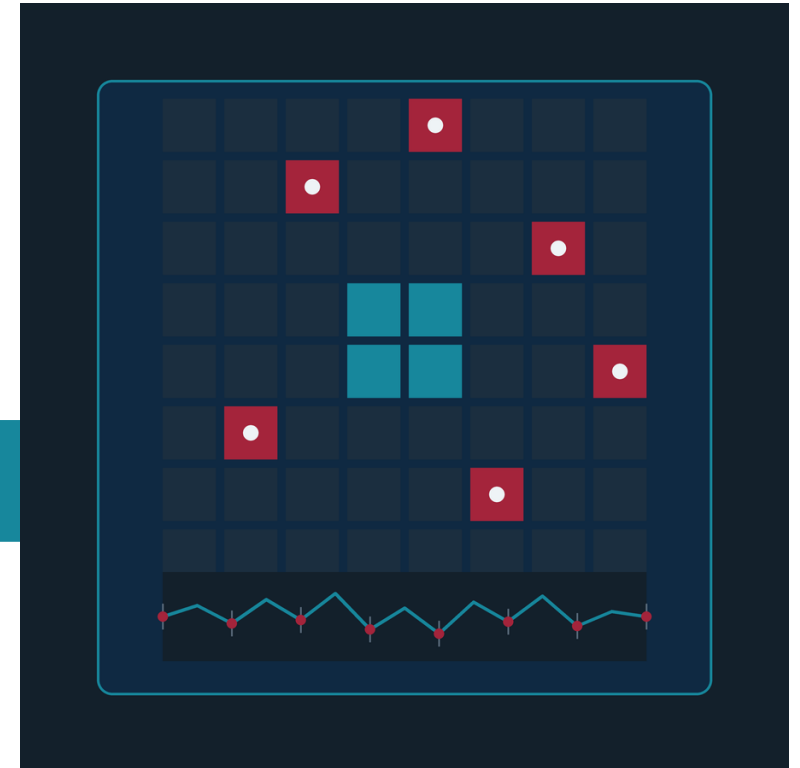
CS2.501

Advanced Computer Architecture

Lecture 2 · Measuring Compute: Benchmarking, Power, and Why Numbers Lie

Suresh Purini · Priyesh Shukla

Monsoon 2026



Instrumented die, schematic — crimson cells mark measurement probes

What we will do in the next 80 minutes

Part 1	What "performance" actually means	Latency, throughput, the Iron Law, honest speedup
Part 2	Summarising with one number	Arithmetic, harmonic, geometric — and when each lies
Part 3	Benchmarks	Suites, SPEC CPU, MLPerf, and benchmarking
Part 4	Power and energy	Watts vs joules, TDP, EDP, energy proportionality
Part 5	Why numbers lie	Variance, measurement bias, baseline games
Part 6	Hands-on lab	Measure, decompose, and then break your own result

Where Lecture 1 left off

We ended with a rule. Today we find out how hard that rule is to obey.

“Never compare clock frequencies. Never compare CPI. Compare wall-clock time on a workload you care about — and report the power.”

Easy to state

Speedup is $T_{\text{old}} / T_{\text{new}}$. Energy is power integrated over that time. Both are ordinary physical quantities with ordinary units.

Hard to obey

Which workload? Against which baseline, tuned by whom? Summarised with which mean? Measured how many times, on a machine in what thermal state?

Routinely got wrong

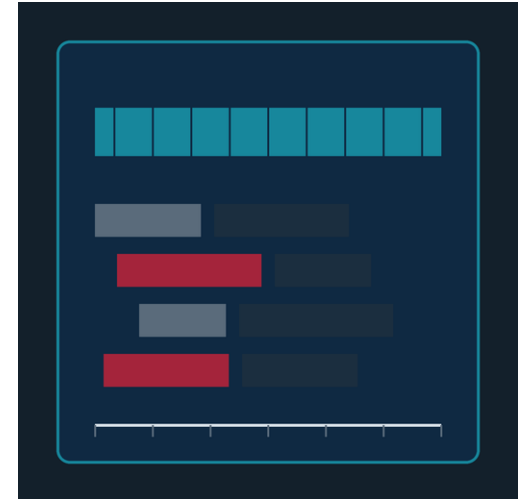
Reproducing studies keep finding reported speedups that shrink, vanish, or reverse once the measurement is controlled.

Takeaway This is a methods lecture. Every unit from here on will be judged by numbers you have to produce and defend.

1

Defining Performance

Before you can measure it, you have to say which question you are asking



Latency and throughput are different questions

Optimising one routinely damages the other

Latency

Time for one task, start to finish. Seconds.

What the user waiting on a page load cares about.

Bought with: higher clock, deeper speculation, bigger caches, prefetching.

Throughput

*Tasks completed per unit time.
Ops/second.*

What the operator billing by the rack cares about.

Bought with: more cores, more copies, batching, SMT, wider vectors.

They trade

Improving one usually costs the other.

Batching raises throughput and raises latency. SMT raises throughput and lowers per-thread latency.

A GPU is a throughput machine that is poor at latency.

Little's Law: $L = \lambda \times W$ concurrency = throughput $\times$ latency

Takeaway SPECspeed measures latency. SPECrate measures throughput. They rank machines differently, and both are correct.

The Iron Law, and the term it hides

CPI is not a property of the microarchitecture — on real workloads most of it is memory

$$\text{CPU time} = \text{IC} \times \text{CPI} \times \text{T_clk}$$

$$\text{CPI} = \text{CPI_core} + \Sigma (\text{MPKI} / 1000) \times \text{penalty}$$

Instruction count

Compiler, ISA, algorithm.

Measure it: `perf stat -e instructions`, or PIN. It is the one term you can count exactly.

CPI_core

Pipeline, issue width, prediction.

The number vendors quote. It is measured on workloads that fit in cache.

The memory term

Misses per kilo-instruction $\times$ penalty.

On a cache-hostile workload this dominates, and the core term stops mattering.

Worked: $\text{CPI_core} = 0.8$, LLC MPKI = 12, DRAM penalty = 250 cycles $\rightarrow$ memory adds $12/1000 \times 250 = 3.0$ CPI. Total CPI = 3.8 — the core contributes 21%.

Takeaway When someone reports IPC, ask on what workload and at what working-set size. IPC without that description is not a number.

Speedup, and the four ways it gets misreported

$S = T_{old} / T_{new}$. Everything after that is bookkeeping people get wrong.

The arithmetic

$S = 2.0$ is “100% faster”, not 200%. “50% faster” is $S = 1.5$. A “50% reduction in time” is $S = 2.0$. Give the ratio, not the adjective.

The baseline

Speedup over an untuned baseline measures the baseline. Compare against the best prior art, optimised with equal effort by someone who wants it to win.

The workload

A speedup on one input is an anecdote. Report the distribution across the suite, and name the worst case as well as the best.

The cost

2× on 8 cores is 25% parallel efficiency. 2× at 3× the power is an energy regression. Speedup alone is half a result.

“X is $S\times$ faster than Y on workload W at input size N, on machine M, over k runs, at P watts.” Any empty slot is a missing result, not a detail.

Which machine is better?

Machine A: 8.2 s at 95 W

Machine B: 6.1 s at 165 W

Time

$$8.2 / 6.1 = 1.34$$

B is 34% faster.

Energy

$$A: 8.2 \times 95 = 779 \text{ J}$$

$$B: 6.1 \times 165 = 1007 \text{ J}$$

A uses 23% less.

EDP (E·T)

$$A: 6\,388 \text{ J}\cdot\text{s}$$

$$B: 6\,140 \text{ J}\cdot\text{s}$$

B, by 4%.

ED²P (E·T²)

$$A: 52\,380$$

$$B: 37\,450$$

B, by 29%.

Four metrics, three different winners. Latency-critical service → B. Battery-powered or energy-billed → A. Fixed power budget in a rack → neither number answers it; you need throughput per watt, which requires the rate, not the single-task latency.

Takeaway Never ask which machine is faster. Ask which is better under the binding constraint — and name the constraint before you look at the data.

2

Summarising Performance

Forty benchmarks, one score — and a choice of mean that decides the winner



Three means, three jobs

Not a matter of taste — the quantity you are averaging determines which one is correct

Arithmetic

$$(1/n) \sum t_i$$

Average raw execution times.

Correct when: the workload mix is equally weighted. $AM \times n$ recovers total time — a real, physical quantity.

Harmonic

$$n / \sum (1/r_i)$$

Average rates — MFLOP/s, IPS, requests/s.

Correct when: equal work per benchmark.
HM of rates = total work / total time.

Geometric

$$(\prod x_i)^{(1/n)}$$

Average normalised ratios — speedups, SPEC ratios.

Correct when: you need a ranking that does not depend on the reference machine.

The test: does mean $\times n$ recover something physical? AM of times does. HM of rates does. GM does not.

Takeaway Averaging speedups with the arithmetic mean is the most common statistical error in architecture papers — and it always flatters the new system.

The same data, three different winners

Two programs, three machines. Watch the ranking move as the summary changes.

	P1	P2	Total time
Machine A	1 s	1000 s	1001 s
Machine B	10 s	100 s	110 s
Machine C	20 s	20 s	40 s

Lower is better. C finishes the whole workload 25× sooner than A.

Arithmetic mean of ratios, normalised to A

A = 1.00 B = 5.05 C = 10.01

→ **A wins**

Arithmetic mean of ratios, normalised to B

A = 5.05 B = 1.00 C = 1.10

→ **B wins**

Geometric mean of ratios, any reference

A = 1.00 B = 1.00 C = 0.63

→ **C wins, and the ranking is stable**

Change the reference machine and the arithmetic mean changes the winner. That alone disqualifies it for normalised data. The geometric mean is immune.

Takeaway But consistent is not the same as correct: the geometric mean calls A and B equal, while B finishes the workload 9× sooner.

Why SPEC reports a geometric mean — and what it costs

One algebraic property buys a stable ranking, and pays for it in meaning

$$\text{GM}(a_i / b_i) = \text{GM}(a_i) / \text{GM}(b_i)$$

The property

The geometric mean of ratios is the ratio of geometric means. Swap the reference machine and every score is rescaled by the same constant — so the ranking cannot move.

What SPEC does

Each benchmark ratio = reference-machine time / your time. The suite score is the geometric mean of those ratios. Base uses one flag set for all benchmarks; peak allows per-benchmark tuning.

Smith's objection

A summary should predict how long the workload takes. The GM predicts nothing: there is no quantity you can multiply it by to recover total time. Consistency without meaning.

What to do

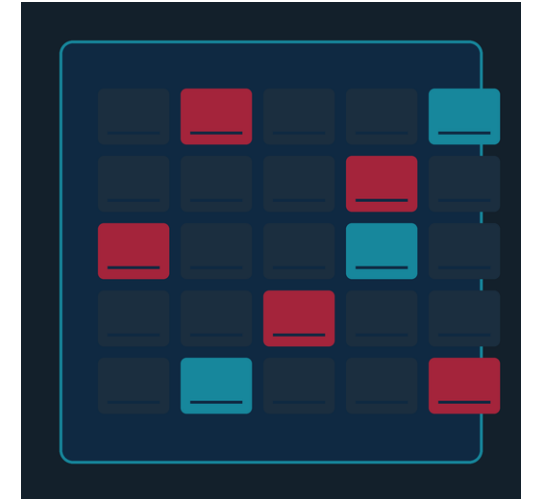
If you know your workload mix, use a weighted arithmetic mean of times with your own weights. And always publish the per-benchmark vector, not only the summary.

Takeaway A single number is a lossy compression of a vector. Report the compression, but always ship the vector alongside it.



Benchmarks

What we agree to measure — and how that agreement gets exploited



A hierarchy of benchmarks

In decreasing order of how much you should believe the result

1

Real applications

The workload you actually run, on your actual input.

Unambiguously relevant. Hard to port, hard to license, slow to run, and impossible for a vendor to publish comparably.

2

Benchmark suites

SPEC CPU, MLPerf, TPC — curated real applications with fixed inputs and run rules.

Comparable across vendors and auditable. Ages badly, and everyone optimises for it.

3

Kernels

LINPACK, STREAM, GEMM, FFT — the inner loop, extracted.

Excellent for isolating one bottleneck. Systematically flatters machines that are good at that one thing.

4

Synthetic and toy

Dhrystone, Whetstone, empty loops, phone benchmark apps.

Cheap and reproducible. Almost no predictive value for real code; the compiler often deletes the work entirely.

Takeaway Real applications beat suites; suites beat kernels; kernels beat synthetic loops; synthetic loops beat opinions.

Anatomy of a SPEC CPU score

The most carefully engineered agreement in the field — worth understanding in detail

Two questions, two suite families

- SPECSpeed — time for one copy of one task. A latency metric.
- SPECrate — N concurrent copies. A throughput metric.
- They rank machines differently: core count wins rate, single-thread wins speed.

Two levels of tuning

- Base — one identical flag set for every benchmark. Mandatory to publish.
- Peak — per-benchmark tuning permitted. Optional, and always higher.
- Comparing your base against a rival's peak is the oldest trick in the results table.

How the score is computed

- $\text{ratio} = \text{reference-machine time} / \text{your time}$.
- Suite score = geometric mean of the ratios. Higher is better.
- Reportable runs use the median of three runs.

What the score does not tell you

- Nothing about your workload unless it resembles the suite.
- Nothing about power — that is SPECpower's job.
- Nothing about tail latency, I/O, or anything off-socket.

Takeaway SPEC's real product is not the number. It is the run rules and the full disclosure of flags, firmware and OS settings that make two numbers comparable at all.

Benchmarks age — and this one is ageing this month

SPEC CPU 2026 shipped in May; CPU 2017 is being retired as we speak

CPU 2017

Released 2017 · retiring 2026

43 benchmarks. 16 GB memory footprint for SPECspeed. Reference machine: a 2006-era Sun Fire V490.

CPU 2026

Released 5 May 2026

52 benchmarks from 38 applications, chosen from over 70 candidates. More than twice the lines of source code.

What changed

The workloads the world runs now

New domains: AI orchestration, EDA, databases, graph analytics. SPECspeed memory raised from 16 GB to 64 GB.

Why suites have to die

Hardware outgrows them. A suite sized for 2017 memory systems stops resolving differences between 2026 machines — everything scores near the top.

Compilers learn them. Once a benchmark is important enough, vendors optimise for its exact source. CPU 2006's libquantum became notorious for this and did not survive into CPU 2017.

Software moves on. A 2017 suite has no answer for what a 2026 datacenter actually does. Hence AI orchestration and graph analytics in the new suite.

Takeaway A benchmark suite has a half-life. When results cluster at the top, the suite has stopped measuring and started certifying.

The benchmark zoo beyond SPEC

Different communities, different agreements about what counts as work

MLPerf

Training and Inference

Time to reach a target accuracy, not raw FLOP/s. Closed division fixes the model; open division allows anything. Read which division a claim is in.

TPC and SPECjbb

Transactions and enterprise Java

Throughput under a latency constraint, with a price-performance metric attached. Closest thing the field has to reporting cost honestly.

PARSEC, SPLASH

Shared-memory parallelism

Built for studying scaling, synchronisation and coherence rather than for ranking products. What Unit 3 will use.

Microbenchmarks

STREAM, Imbench, pointer chase
Measure one machine parameter precisely: bandwidth, latency, TLB reach. Essential for models like Roofline. Never a product claim.

Choosing one

Match the benchmark to the claim you want to make.

A microbenchmark supports a statement about a mechanism — “our prefetcher raises achieved bandwidth by 18%”. A suite supports a statement about a machine — “this part is 1.3× on integer throughput”. Using a microbenchmark to make a machine-level claim is the most common category error in student projects, and in a fair number of published papers.

Takeaway Ask of any suite: who curated it, who pays for it, and what does it structurally fail to measure?

Benchmarketing

Every benchmark that becomes important enough will be optimised for rather than with

Gaming the compiler

- Flags tuned per benchmark, then reported against a rival's base result.
- Source-recognition: the compiler detects a known benchmark's structure and applies a transformation no real program would receive.
- Autoparallelisation enabled only where it happens to help.

Gaming the platform

- Detect the benchmark process, then raise power limits or lock turbo — the mobile-SoC defeat device.
- Report on a chassis, cooling or memory configuration nobody can buy.
- Quote peak theoretical FLOP/s that no kernel has ever achieved.

Gaming the presentation

- Truncated y-axis; a 3% gap drawn as a chasm.
- Best-of-N against the rival's mean.
- Geometric mean quoted when it flatters, total time when that flatters instead.
- “Up to” — the two words that turn a maximum into a headline.

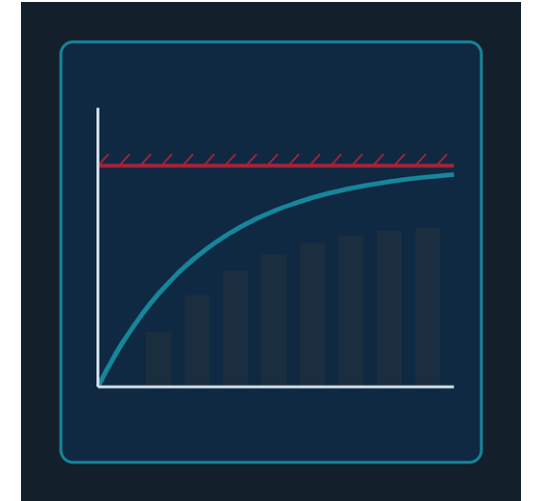
None of this requires anyone to lie. Every number can be individually true and the conclusion still false.

Takeaway Goodhart's law, in silicon: when a measure becomes a target, it stops being a good measure.

4

Power and Energy

The binding constraint since 2004 — and the one most papers still omit



Power, energy, and TDP are three different things

Confusing them is the most common error in performance reporting, and the easiest to avoid

Power — watts

$$P = \alpha CV^2f + V \cdot I_{\text{leak}}$$

An instantaneous rate. It is a constraint: it sets what your cooling, your voltage regulators and your rack can sustain. It is not a cost.

Energy — joules

$$E = \int P \, dt$$

The integral over the run. This is what the electricity bill and the battery actually track. Halving power while doubling runtime saves nothing.

TDP — neither

A thermal design specification

The sustained heat the cooling solution must remove. Not peak power, not average power, not measured power. Modern parts exceed it in short bursts by design.

Never compute energy as TDP × runtime.

It is a nameplate figure, set by marketing and thermal engineering together, and it is defined differently by every vendor and revised by firmware. It will be wrong in both directions: idle parts draw far less, bursty parts draw far more. Measure the energy counters instead — you will do exactly this in the lab.

Takeaway Power is what you are allowed to spend. Energy is what you actually spent. Report the second, respect the first.

Efficiency metrics, and the one you cannot game

Each metric encodes a different opinion about how much you value time

Perf per watt

$$\text{ops/s} / W = \text{ops/J}$$

Algebraically identical to the inverse of energy per operation. Maximising it means minimising joules per unit of work.

Energy per op

$$\text{pJ/op, J/token, J/frame}$$

The cleanest number in the field. Comparable across wildly different machines, and the one accelerator designers actually optimise.

EDP

$$E \cdot T$$

Values time and energy equally. But $\text{EDP} \propto f$ under DVFS — so it is minimised by simply running as slowly as possible.

ED²P

$$E \cdot T^2$$

Weights time more heavily. Under the same scaling ED²P is constant — so it is invariant to pure DVFS and isolates architectural efficiency.

Why ED²P is the circuit designer's metric

Under classical DVFS, $V \propto f$, so $E \propto CV^2 \propto f^2$ and $T \propto 1/f$.

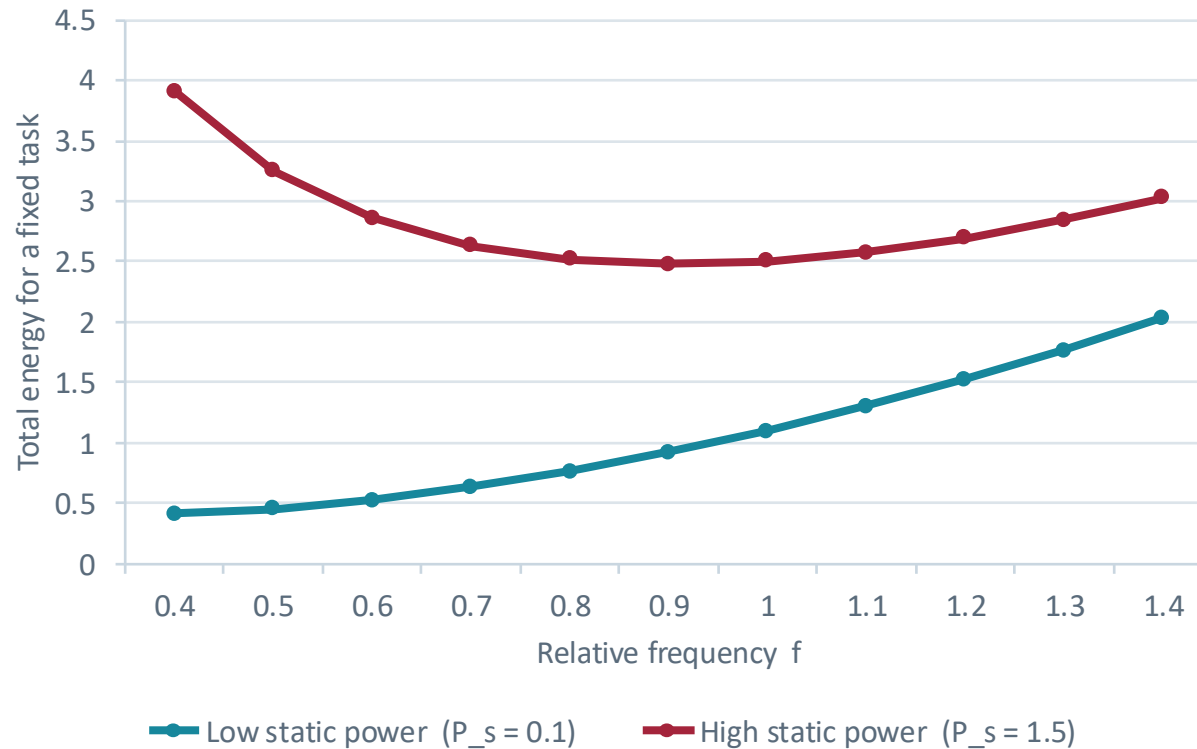
Then $\text{EDP} \propto f^2 \cdot f^{-1} = f$ — minimised at $f \rightarrow 0$, which rewards doing nothing slowly.

And $\text{ED}^2\text{P} \propto f^2 \cdot f^{-2} = \text{constant}$ — unchanged by voltage scaling, so any improvement in ED²P came from the design, not the clock.

Takeaway Pick the metric before you see the data. Choosing it afterwards is how honest people produce dishonest conclusions.

Race to idle, or slow down?

The answer is not a matter of principle — it depends on how much static power you pay



The model

Fixed work. Dynamic power $\propto f^3$ (since $V \propto f$), so $E(f) = f^2 + P_s / f$, with the machine idling at negligible power once finished.

The optimum

$$dE/df = 0 \rightarrow f^* = (P_s / 2)^{1/3}$$

The best frequency rises as the cube root of static power.

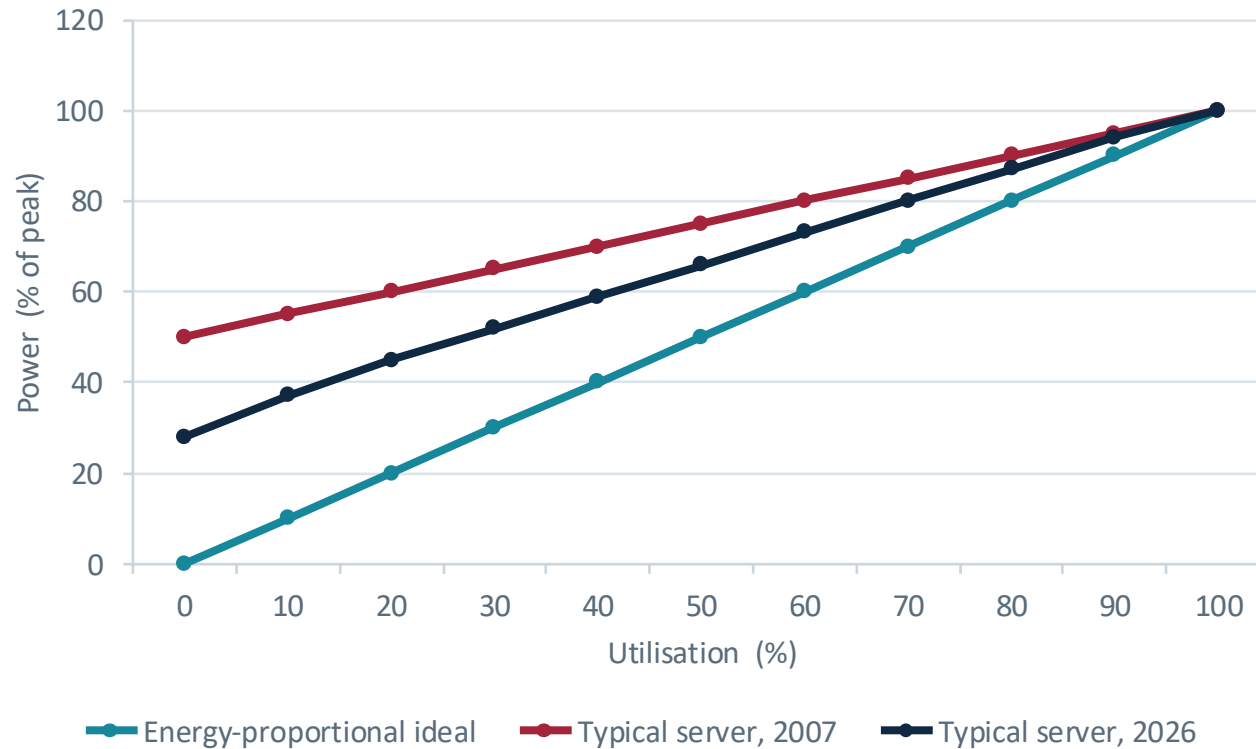
So

Leaky chip $\rightarrow$ race to idle. Low-leakage chip $\rightarrow$ slow down and stay there.

Takeaway Every DVFS governor is an implementation of this trade-off, and the right answer moved as leakage grew after Dennard scaling ended.

Energy proportionality

Servers spend their lives at low utilisation, where they are least efficient



The observation

Barroso and Hölzle, 2007: servers run most often between 10% and 50% utilisation — exactly the band where efficiency is worst.

Why it matters

A machine at 30% load drawing 55% of peak power is wasting most of its energy on being switched on.

The responses

Deep sleep states, consolidation, heterogeneous cores, and serverless billing.

Takeaway Peak efficiency is the wrong figure of merit for a fleet. Efficiency at the utilisation you actually run at is the right one.

How to actually measure power

Three instruments, three different boundaries — and each one lies in its own way

RAPL (CPU)

- Energy counters in the processor. Domains: package, cores, uncore, DRAM.
- `perf stat -a -e power/energy-pkg/`
- Caveats: package only, not the board. Counters wrap in about a minute. Root-only since the PLATYPUS side channel.

nvidia-smi (GPU)

- `nvidia-smi --query-gpu=power.draw --format=csv -lms 100`
- Board-level draw, sampled — not integrated energy. You integrate it yourself.
- Caveats: the sampling period is coarse relative to kernel launches, and the reading lags the true draw.

Wall meter (system)

- The only measurement that includes fans, drives and PSU losses.
- Ground truth for a whole-system claim; required for SPECpower reporting.
- Caveats: PSU efficiency varies with load; too slow to attribute energy to one kernel.

State the boundary with every power number.

A 3× improvement in package energy can be a 1.2× improvement at the wall once DRAM, fans and PSU losses are counted — and a regression once you include the host CPU that had to feed the accelerator. Most accelerator papers report the accelerator. Ask what the rest of the machine was doing.

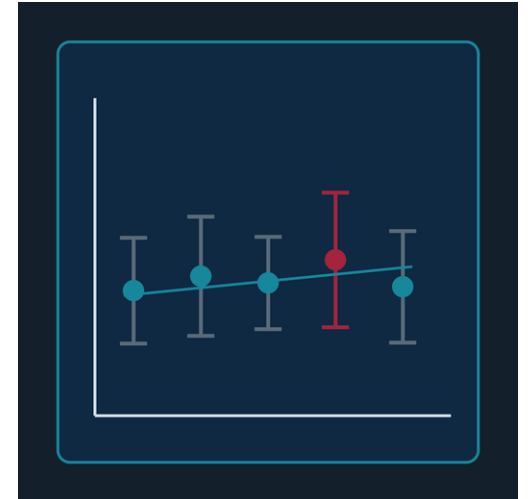
Takeaway You will use RAPL in the lab in fifteen minutes. Note its boundary in your report before you quote a single joule.

5

Why Numbers Lie

The machine under your measurement is not the machine in your model

PART 5



Where the variance comes from

Your binary is deterministic. Almost nothing else about the experiment is.

Frequency

- Turbo raises the clock, then thermals lower it mid-run.
- Governor changes under load.
- Neighbouring cores steal power budget.

Placement

- NUMA node of the allocation.
- Which physical core; which SMT sibling.
- Huge pages granted or not.
- Page colouring changes cache conflicts.

State

- Cold caches, cold TLB, cold branch predictor.
- Filesystem cache warm from the previous run.
- JIT warmup; lazy dynamic linking.

Neighbours

- Other tenants on the socket in a shared machine.
- Interrupts, timers, the scheduler itself.
- In the cloud, a physical host you cannot see.

Controlling what you can

`cpupower frequency-set -g performance` fix the governor · `echo 1 > .../intel_pstate/no_turbo` remove turbo variability

`taskset -c 2` pin the core · `numactl --membind` fix the memory node · `setarch -R` disable ASLR

Then warm up, discard the first runs, and repeat at least fifteen times.

Takeaway Controlling the environment does not make your result more true. It makes it possible to tell whether it is true.

Producing wrong data without doing anything obviously wrong

Measurement bias: a systematic error you cannot remove by averaging

Changing the size of an environment variable, or the order objects are linked, can reverse the conclusion of a compiler-optimisation experiment.

What they did

Mytkowicz and colleagues measured the same benchmarks across many random link orders and environment sizes — variables no one thinks of as experimental parameters.

What they found

The induced spread was large enough that O2 could be shown to beat O3, or the reverse, depending purely on where the code and stack happened to land in memory.

Why

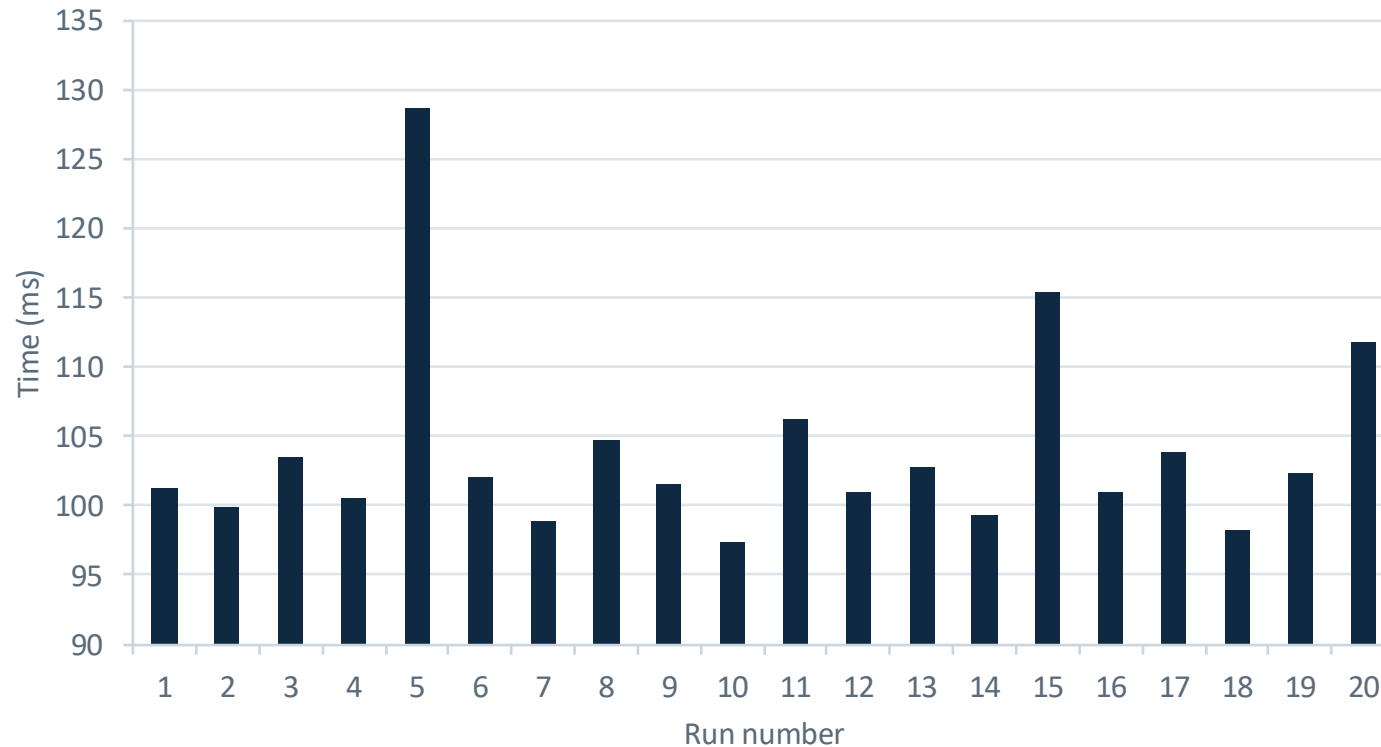
Layout changes alignment, which changes instruction-cache and branch-predictor conflicts. Bias is systematic, so repeating the same configuration never reveals it.

The fix: randomise the setup, not just the runs. Sample over link orders, environment sizes and layouts, and report the distribution that produces.

Takeaway If your effect is smaller than your layout noise, you have not measured your effect. You will test this on your own code in the lab.

Report distributions, not points

Twenty runs of one unchanged binary on one quiet machine



The statistics

min 97.4 median 101.8

mean 104.0 max 128.6

The mean sits above the median: timing distributions have a right tail, never a left one.

The free speedup

Report the minimum instead of the median and you have manufactured a 4.5% improvement from nothing.

What to report

Median, IQR, n, and the raw runs in an appendix.

Takeaway There is a floor set by the hardware and no ceiling set by anything. That is why the median, not the mean, is the honest centre.

Honest-looking lies

Each of these has appeared in a published architecture paper, repeatedly

The weak baseline

Compared against an unoptimised reference the authors wrote themselves, rather than the best available implementation tuned by someone motivated to win.

Unequal effort

Six months of tuning on the new system, an afternoon on the baseline. The speedup measures the effort asymmetry.

The convenient input

Results on the one input size that fits the new cache, with the others quietly absent from the table.

Strong-scaling sleight of hand

Fixed total work makes efficiency fall with core count. Growing the problem with the machine hides it. Say which you did.

Extrapolation

Measured to 16 cores, plotted to 1024. Amdahl's law is a ceiling, not a trendline to extend.

The missing power column

A speedup obtained by raising the power limit is a purchase, not a design contribution.

Takeaway Read your own results the way a hostile reviewer would, before a hostile reviewer does.

A checklist for an honest performance claim

Ten lines. If you cannot fill them all in, you do not yet have a result.

- 1 The workload, by name, version and input size.
- 2 The baseline, and who tuned it and how hard.
- 3 The machine: part number, memory, firmware, OS, compiler and flags.
- 4 The number of runs, and which statistic you report.
- 5 The dispersion: IQR, standard deviation, or the raw runs.
- 6 The power boundary, and the energy figure that goes with the time.
- 7 Which term of the Iron Law moved, and evidence that it did.
- 8 The worst case, not only the geometric mean.
- 9 The threats to validity you know about and did not fully eliminate.
- 10 Enough detail that a stranger could reproduce it — scripts, not prose.

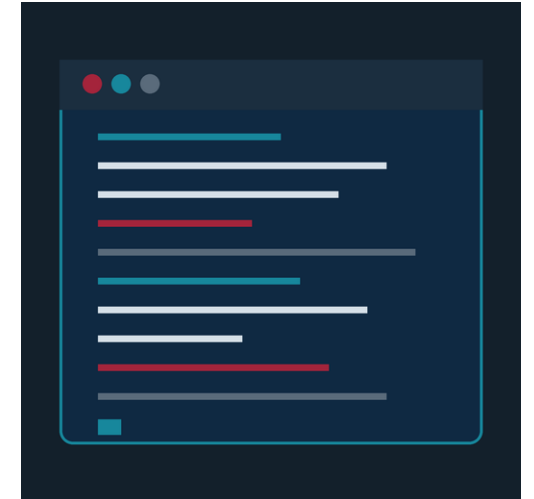
Takeaway This is the standard your homeworks, your project report and the papers we read in the tutorial will all be held to.

6

Hands-on Lab

Measure a real kernel, decompose it, price it in joules — then break your own result

PART 6



Take home Lab — what you are about to measure

Five steps. Work in pairs. The last step is the one that matters.

The subject

- mm.c — a square matrix multiply with three loop orders: ijk, ikj and a blocked variant.
- Identical instruction counts to within a fraction of a percent; wildly different run times.
- Everything you need is in the course repository under lab02/.
- Sizes 512 and 1024 both run in well under a second per repetition.

```
# one-time setup
sudo apt install linux-tools-$(uname -r)
git clone <course-repo> && cd lab02
make                # builds mm, ijk / ikj / blocked

# quiet the machine
sudo cpupower frequency-set -g performance
echo 1 | sudo tee \
    /sys/devices/system/cpu/intel_pstate/no_turbo
# check the energy counters are readable
perf stat -a -e power/energy-pkg/ sleep 1
```

If any of that fails.

On Apple silicon, substitute powermetrics for RAPL and skip the no_turbo step. On a shared or virtualised machine the energy counters are usually unavailable — pair up with someone on bare metal, and note the limitation in your report rather than working around it. Being unable to measure something is a finding, and reporting it is part of the exercise.

Step 1 — build a timer you can trust

Four minutes. Most bad measurements are already lost at this stage.

```
#include <time.h>
static double now(void) {
    struct timespec ts;
    clock_gettime(CLOCK_MONOTONIC, &ts);
    return ts.tv_sec + 1e-9 * ts.tv_nsec;
}
warmup(); // discard
for (int r = 0; r < 15; r++) {
    double t0 = now(); kernel();
    record(now() - t0);
}
```

The rules

- CLOCK_MONOTONIC, never gettimeofday — the wall clock can step backwards.
- Size the work so one repetition takes at least 100 ms.
- Warm up, then discard the first runs.
- At least 15 repetitions. Record every one; summarise later, not in the loop.
- Print the raw times. You cannot recover a distribution you never saved.

Do this now

```
taskset -c 2 setarch -R ./mm ijk 1024
```

Record the median and the interquartile range. Then run it again on an unpinned core with ASLR enabled and compare the spread. Write both numbers down — you will need them in Step 5.

Step 2 — decompose it with the Iron Law

Five minutes. Find out which term actually moved.

```
perf stat -r 15 \  
  -e cycles,instructions,branch-misses,\  
  LLC-load-misses,dTLB-load-misses \  
  taskset -c 2 ./mm ijk 1024  
  
# repeat for: ikj, blocked
```

Derive

- $\text{IPC} = \text{instructions} / \text{cycles}$
- $\text{CPI} = 1 / \text{IPC}$
- $\text{MPKI} = 1000 \times \text{LLC-load-misses} / \text{instructions}$

Expect

Instruction counts within ~1% across all three variants. Run times differing by several times over.

Conclude

IC did not move, so the entire effect lives in CPI — and the MPKI column tells you which part of CPI.

Answer this

Using your measured MPKI and an assumed 250-cycle penalty, how much of the CPI gap does the memory term explain?

Step 3 — price it in joules

Four minutes. The same three kernels, now measured in energy.

```
perf stat -a -r 5 \  
-e power/energy-pkg/,power/energy-ram/ \  
taskset -c 2 ./mm blocked 1024  
  
# or read the counter directly:  
cat /sys/class/powercap/intel-rapl:0/energy_uj  
  
# GPU, if you have one:  
nvidia-smi --query-gpu=power.draw \  
--format=csv -lms 100
```

Compute and tabulate

- E in joules, T in seconds
- Average power $P = E / T$
- $EDP = E \cdot T$
- $ED^2P = E \cdot T^2$
- Energy per useful FLOP

The question to answer

Does the fastest variant also use the least energy? It usually does, because it finishes sooner and static power is paid per second — but check the average power column before you assert it. Then state your RAPL boundary explicitly: package plus DRAM is not the wall, and your report must say so.

Step 4 — summarise the mini-suite

Four minutes. Now the means from Part 2 stop being abstract.

```
import statistics as st

base = [...]    # median times, ijk
opt  = [...]    # median times, blocked
r = [b / o for b, o in zip(base, opt)]

print('AM of ratios      ', st.mean(r))
print('GM of ratios      ', st.geometric_mean(r))
print('Total-time        ', sum(base) / sum(opt))
```

Run five sizes

- 256, 384, 512, 768, 1024
- Median of 15 runs at each size
- Then compute all three summaries
- They will not agree. The gap between them is the entire point of Part 2.

Which of the three is your honest headline number?

Defend the choice in one sentence in your report. There is a defensible answer for each of them — what is not defensible is choosing after you have seen which is largest.

Step 5 — break your own result

Five minutes. Reproduce measurement bias on the machine in front of you.

```
# sweep the environment size
for i in $(seq 0 30); do
  PAD=$(printf 'x%.0s' $(seq 1 $((i*64)))) \
  perf stat -r 5 -x, -e cycles \
    taskset -c 2 ./mm ikj 512 2>&1 | tail -1
done

# then relink in a different object order
make shuffle-link && ./mm ikj 512
```

What to look for

- The spread across environment sizes, with the binary completely unchanged.
- Whether that spread is larger or smaller than the ikj-vs-blocked effect from Step 2.
- Whether the ranking of two close variants ever flips.

If the layout noise is comparable to your effect, you have not measured your effect. Report the noise distribution alongside the result, or do not report the result.

Takeaway This is the step to remember in six months, when your project produces a 4% speedup and you have to decide whether you believe it.

What to submit

Two pages, due before the next tutorial. Scripts in the repository, not pasted into prose.

The report

- A table: three variants × five sizes, each with median time, IQR, IPC, MPKI, energy and EDP.
- Your headline speedup, written in the full sentence form from Part 1.
- The layout-noise distribution from Step 5, and a statement of whether your effect survives it.
- One paragraph justifying your choice of summary statistic.
- One paragraph on threats to validity you could not eliminate.

The rubric

Measurement protocol

Pinning, repetitions, warmup, correct timer, raw data retained.

Iron Law decomposition

Correct derivation of IPC and MPKI; the right conclusion about which term moved.

Energy analysis

Correct EDP and ED^2P ; boundary of the RAPL measurement stated.

Honesty about limits

Step 5 taken seriously rather than reported as a formality.

Before the next lecture

Read

- Hennessy & Patterson, 6th ed., §1.6–1.9 — performance, power and cost.
- Fleming & Wallace, CACM 1986, and J. E. Smith, CACM 1988. Two short papers that disagree; decide which you side with.
- Mytkowicz et al., “Producing Wrong Data Without Doing Anything Obviously Wrong!”, ASPLOS 2009.

Optional but recommended

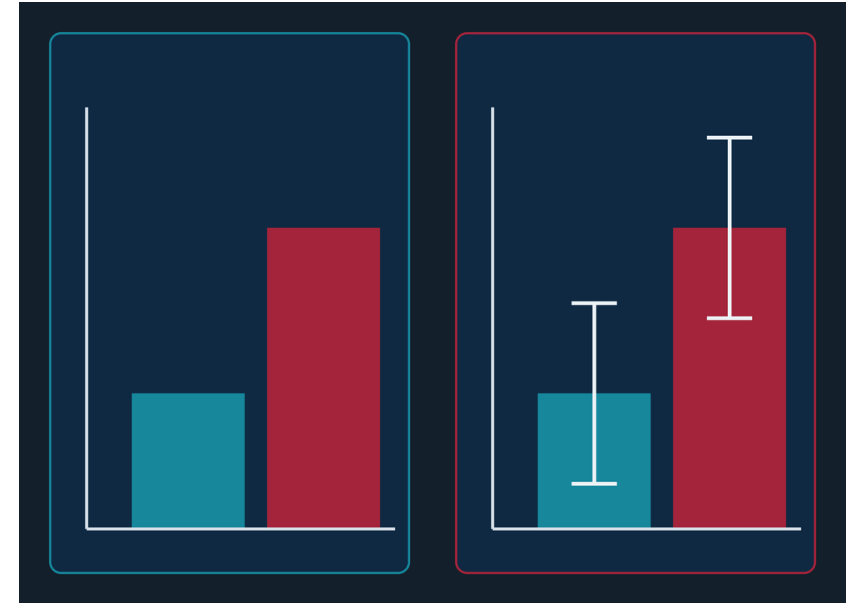
- Barroso & Hölzle, “The Case for Energy-Proportional Computing”, IEEE Computer, 2007.
- Williams, Waterman & Patterson, “Roofline”, CACM 2009 — the model we will use in Unit 2.
- Hoefer & Belli, “Scientific Benchmarking of Parallel Computing Systems”, SC 2015 — twelve rules for reporting.

Come to the tutorial ready to answer

- A paper reports a $1.8\times$ geometric-mean speedup across 12 benchmarks, with a $0.7\times$ worst case. What is the single most important extra number to ask for, and why?
- Your kernel runs $1.5\times$ faster at $2\times$ the power. Under what constraint is that an improvement, and under what constraint is it a regression?
- You measure a 3% speedup with an interquartile range of 4%. What do you do next — and what may you not yet claim?

The one sentence to remember

**A number without a workload,
a baseline, a variance and
a power figure is not a
measurement. It is a marketing
claim with a decimal point.**



The same two results, reported two ways